

Lean Ontology Matching: A Complexity Theoretic Approach

Albert Meroño-Peñuela^{1,2}, Steven de Rooij^{1,3}, Stefan Schlobach¹, Christophe Guéret², and Frank van Harmelen¹

¹ Department of Computer Science, VU University Amsterdam, NL
`albert.merono@vu.nl`

² Data Archiving and Networked Services, KNAW, NL

³ Department of Computer Science, University of Amsterdam, NL

Abstract. Ontology matching is an essential task for achieving interoperability in the Semantic Web. In this paper we present the technique of Lean Ontology Matching (LeanOM), which exploits Kolmogorov complexity to quantify the information shared between two ontology entities. Our hypothesis is that this approach suits well ontologies that are lexically and structurally similar, like evolving versions of one ontology. LeanOM is generic, unsupervised, domain agnostic, and lean due to its simplicity. We run LeanOM with different standard OAEI datasets and tests, discussing benefits, disadvantages, and suitability of this novel method.

Keywords: Ontology Alignment, Ontology Matching, Graph Compression

1 Introduction

Ontologies are models of the world that describe its properties: things, individuals, places, ... As every model, these ontologies are grounded on the timely and geolocally located perception of the reality they model. The result is that a single ontology, properly maintained, will evolve over time. Some concepts may drift [52], be deleted or be created. The Universal Decimal Classification (UDC) is a living example of such an evolution. Several times per year the content of this classification scheme, used by librarians, is updated to better take into account knowledge fields and topics that are appearing, fading out of fashion or being created. Another example lies in the historical census data of the Netherlands. This dataset⁴ comprises the population, occupation and housing censuses performed once every ten years between 1795 and 1971. In each of these, the classifications used to model population, occupations or house types were updated to reflect changes in their domains.

The digital humanities project CEDAR⁵ aims at improving the usability of the historical Dutch census data by enabling longitudinal querying over all the

⁴Available at <http://www.volkstelling.nl/nl/index.html>

⁵See <http://www.cedar-project.nl/>

census years. Up to now, such a task involves several hours of the time of a census expert, historian by training, to dig in the data and take out all the relevant numbers. The most challenging aspect of this task is related to the evolution of the underlying ontology. Looking for the total number of “shoemakers” in the period 1795–1940 in a certain municipality, for instance, involves keeping track of all the evolutions this concept has been through over the years, from the person nailing down together pieces of leather to the businessman running a company.

Establishing these mappings is a challenging task that involves a lot of expert knowledge. (Semi-)automated ontology alignment tools are not expected to perform well on such knowledge intensive problems. However, they could be used to facilitate the task of historians by suggesting mappings to look at. These suggestions should be provided quickly, just as a guideline for the manual work to follow. Yet, the majority of the ontology alignments that have been produced over the last couple of years have been optimised for giving the best possible mappings at the cost of spending more time computing them.

In this paper, we introduce an innovative approach to the problem of ontology alignment called *Lean Ontology Matching* (LeanOM). This technique aims at being fast and simple to compute, leveraging the relatively small amount of changes observed between two successive versions of the same ontology. In order to detect such changes, we measure the distance between the versions using Kolmogorov complexity. Because this measure is not computable in practice, we approximate it using a compression algorithm.

Rather than an *ad hoc* approach, we see this as a fundamental approach to the notion of similarity between concepts: compression is not just one way of summarising, it is the most dense representation of the complete information about some object. The typical fundamental problems of using compression to approximate Kolmogorov complexity are that the compressed result: 1) might lose information (semantics, etc) and 2) might not be minimal with respect to the theoretical compression value. The latter is not an issue for LeanOM, but the former is critical: the performances of LeanOM are expected to be strongly tied to the serialization made of the concepts.

This work aims at assessing whether Kolmogorov complexity can be used as a lean way to compute alignments between successive versions of an ontology. In particular, the research questions tackled by this work are:

- How can we provide a rich information serialization of a concept?
- How rich shall the description of a concept be to better account for its information complexity?
- Can a change in Kolmogorov complexity of the description of a concept be used to detect changes in its semantics?
- Is an ontology matcher tracking changes in information complexity good enough to monitor the evolution of a single ontology?

The remaining of this paper is organised as follows. In Section 2 we look at the state of the art of ontology matching and motivate our approach, which uses Normalized Compression Distance as a distance measure based on Kolmogorov complexity. This distance is introduced in Section 3. It is subsequently used in the

implementation of the Lean Ontology Matcher (LeanOM), which is described in Section 4. Our experimental results are presented in Section 5. We finally conclude with a discussion of current results (Section 6) and future work (Section 7).

2 Related work

We see our work as being essentially related to the topic of (agnostic) ontology matching, ontology change monitoring, and learning by data compression. Some highlights of these three bodies of work are hereafter introduced. Although this paper is about compression, there is no strong connection between LeanOM and approaches such as HDT [10]: in our work, compression is not used for the purpose of efficient storage or transmission, but rather as a proxy to measure the information complexity of a concept. Prior applications of this kind are also discussed below.

2.1 (Agnostic) Ontology Matching

Literature on ontology matching is huge⁶. It comprises four essential methods: terminological, structure-based, semantic-based and instance-based matching [8]. Terminological methods exploit lexical data in ontologies, like labels, identifiers or comments, to discover concept mappings. Structure-based methods use the internal or external structure of concepts, like subclasses or superclasses, to deduce specific relations between concepts. Semantic-based methods use generic or domain specific rules and/or background information to find correspondences between ontologies. Instance-based methods use concept extensions (the set of instances that belong to that concept) to align ontologies.

Current ontology matchers usually combine terminological, structure-based and semantic-based matching, and it is hard to find pure approaches using only one of these components. Several methods exploit lexical and structural characteristics of two ontologies to calculate similarity between them [17,20,34]. Maybe a pure structural method is that proposed by Mascardi et al. [35], in which a matching with upper ontologies is performed. Krishna Kumar et al. develop a semantic-based approach, using description logic based bridging axioms between the ontologies in order to match them [27].

Regarding instance-based matching, a straightforward approach is to measure the *common extension* of the concepts, that is, the set of objects that are simultaneously classified by both concepts [18,19]. But Jaccard-like similarity measures are the common approach to extensional matching ([28,50]). Other variants use the DICE similarity [47], or the Jensen-Shannon distance [54]. Zaiss [55] describes two other instance-based matchings methods, one of which consists in aggregating instances and properties of the concepts that are to be mapped. Similarly, Wang *et.al.* [51] present a classifier that assigns pairs of source and target concepts to matches and non-matches. Other approaches ([48,32]) use

⁶See, *e.g.*, <http://www.ontologymatching.org/publications.html> and [8]

multiple Machine Learning techniques for weighting features of similarities between classes of instances. In their survey [5] Choi et al reported that 4 out of 9 systems they studied used instance-based methods.

Recent approaches, though, apply combinations of multiple matching techniques [29], often including an instance-matching component. This is the common trend in modern matching systems, as reported by a recent survey by Shvaiko and Euzenat [44].

2.2 Concept Drift

A decent overview about concept change is given in previous work [53], as well as a novel theory for the semantics of change. Here we only remind the reader of some existing work.

Concept drift is widely recognised as an important issue in Semantic Web applications⁷. However, there is no generally accepted way of dealing with changing concepts over time yet, although there is some related research ([14,33,36]).

For the Machine Learning literature [49], concept drift means that the statistical properties of the target variable, which a classifier is trying to predict, change over time. In [25] concept drift is discussed due to evolving notions over time in technical standards. In [9], Fanizzi et al. describe an unsupervised learning method to perform conceptual clustering, detecting concept drift by finding new individuals that fall apart from existing clusters.

In the Semantic Web community, *ontology evolution and change management* refer to the “problem of deciding the modifications to perform upon an ontology in response to a certain need for change as well as the implementation of these modifications and the management of their effects in depending data, services, applications, agents or other elements” [11]. Existing research addresses this problem at the macro ontological level, studying the effect of certain change operations over the ontology elements, as well as the interoperability between different versions over time [15]. Change detection for individual concepts is relevant for the latter [37]. Different techniques for detecting such changes have been developed, based on studying concept formal definitions [38,42], concept syntax [24], and intension in different ontology versions [16], as well as interpreting the effects of such changes [23]. A whole body of research in ontology change management investigates change management in collaborative environments [22,45].

Recently, in Description Logics some approaches to determine changes in meaning through ontology *diffs* in versioned ontologies have been investigated. Gonçalves *et al.* [13] claim that a purely syntactic analysis of changes is insufficient, and advocate for a *diff* method based on the distinction between logically

⁷E.g., see the fact that the topic is mentioned in both the discussion about the design of SKOS (<http://www.w3.org/2001/sw/wiki/SKOS/Issues/ConceptEvolution>) and the previous version of OWL (<http://www.w3.org/TR/2004/REC-owl-guide/>), while there is no fine-grained solution yet in OWL2.

effectual and ineffectual changes. This work is extended considering also ontology entailment [12]. Additionally, there is also a lot of related work on measures for concept similarity [21].

2.3 Learning by Data Compression

Besides their obvious uses, data compression algorithms have also proven to be very effective tools in a variety of learning tasks, especially clustering and classification problems. Some early results are reported in [46,1]. A more solid theoretical underpinning for these methods was developed on the basis of Kolmogorov complexity [2]; this later led to the development of the Normalized Information Distance [30] which was applied in various settings [3,43,6]. Compression-based inference has not previously been applied to linked data.

3 Normalized Compression Distance

The Normalized Compression Distance between two data objects x and y , denoted $NCD(x, y)$, is a distance measure motivated using the concept of Kolmogorov complexity. This is a theoretical, but natural definition of the amount of information present in an individual data object. In this section we first explain what Kolmogorov complexity is, and then show how it can be used to define a universal distance measure. Finally we describe how we approximate this theoretical construct to become suitable for practical use.

3.1 Kolmogorov Complexity

How much information does a data object contain? To answer this question using Shannon’s Information Theory, we need to assume that the data is a realisation of a known probability distribution. In practice, this is often unfeasible: who can claim to be able to define the distribution that underlies, say, the works of William Shakespeare? Nevertheless, intuitively a sequence of a thousand zeroes contains less information than a sequence of a thousand coin flip results (unless these miraculously all landed same side up). Kolmogorov proposed a different solution: the *Kolmogorov complexity* $K(x)$ of a binary sequence x is, roughly put, the length of the shortest computer program that will output x . To make this more precise, first fix a universal Turing machine U that reads programs in prefix-free representation — this means that programs for U are self-delimiting; no special markers are allowed to signal the end of the program on the input tape. The Kolmogorov complexity $K_U(x)$ of x with respect to U is then the length of the shortest U -program that outputs x and then halts. This universal machine U may be interpreted as the “language” in which the structural properties of x are described; by changing U the Kolmogorov complexity changes so this definition appears arbitrary at first glance. However, one can prove that for any two universal Turing machines U and V there is a constant c such that for all data x we have $K_U(x) - K_V(x) \leq c$. Thus, as long as the data object x

is sufficiently complex, the impact of changing the universal Turing machine is minor. Consequently, the next step is to fix a single universal Turing machine U that is used in all subsequent discussion; in the remainder of the paper we omit U from the subscript.

An unfortunate property of Kolmogorov complexity is that it is not computable: it is impossible in general to find the shortest program for a data object x , because given a candidate shortest program there may be other, shorter, programs that do not halt; because of the halting problem it is impossible to tell whether or not they will eventually output x and halt. Below we first use this theoretical construct to define a universal distance measure, and then discuss how to make this practical.

A more comprehensive introduction to Kolmogorov complexity is given in the textbook [31].

3.2 Normalized Information Distance

Many concepts from classical information theory have an analogue for individual sequences based on Kolmogorov complexity. An example is mutual information. The *algorithmic mutual information* $I(x : y)$ measures the amount of information that is shared by data objects x and y . It can be defined as $I(x : y) := K(x) + K(y) - K(x, y)$, intuitively the number of bits saved by using a single program to construct both x and y rather than two separate programs. The algorithmic mutual information is in itself a measure of similarity. However, it does not reflect the intuition that large objects (in the sense of long strings) that differ by a tiny part are intuitively closer than tiny objects that differ by the same amount. For example, we would naturally consider two huge ontologies that are identical except for a single concept to be “more similar” than those two concepts by themselves are. To avoid this problem, Cilibrasi and Vitányi define a distance measure called the *Normalized Information Distance* (NID) [30], see also [7]. Written in terms of mutual information, the NID can be defined as follows:

$$\text{NID}(x, y) := 1 - \frac{I(x : y)}{\max\{K(x), K(y)\}}.$$

Since the mutual information between x and y is nonnegative and cannot be much larger than the complexity of either object, the NID is a number between 0 and 1; moreover it is a distance metric: it satisfies reflexivity: $\text{NID}(x, x) = 0$, symmetry: $\text{NID}(x, y) = \text{NID}(y, x)$, and the triangle inequality: $\text{NID}(x, z) \leq \text{NID}(x, y) + \text{NID}(y, z)$. Well, almost: because some Turing machines happen to have a relatively efficient representation on the universal machine compared to others, small violations of these inequalities may occur, especially when the considered objects have low complexity. The extent to which the metric relations can be violated is quantified in [30, Lemma V.4].

The NID is a useful and important distance because it is *universal* in the sense that it is sensitive to *all* computable relations between its inputs. This is both a strength and a weakness: on the one hand, it removes the possibility to

emphasize some features of the inputs over others, as can be done when using a distance that is hand-crafted for a particular application. On the other hand it may detect similarities that are not even captured by any of the features that the user considered.

3.3 Approximation using data compression

As mentioned, the Kolmogorov complexity is uncomputable. Nevertheless, the theory has been successfully applied in a substantial number of practical circumstances by approximating Kolmogorov complexity using general purpose data compression software; for related work see Section 2.3.

It is impossible to know whether or not the approximation is close to the true complexity of the data, so the effectiveness of these applications must always be verified empirically. In particular, the method will not work in cases where the used compressor is not suitable for the considered data and fails to exploit its relevant patterns and regularities. General purpose data compression software is good at exploiting repeating substrings in the data, a model that manages to capture the relevant statistics in a surprisingly wide variety of domains; moreover existing compression software is optimised for speed and often yields a very good tradeoff between performance and modelling power.

To calculate the mutual information $I(x : y)$, we need the Kolmogorov complexity of the pair $K(x, y)$. To approximate this we use the fact that the complexity of the pair is very close to the complexity of the concatenation, $K(x, y) \approx K(xy)$. Subsequently replacing Kolmogorov complexity $K(x)$ of an object x with its compressed length $L(x)$, we obtain a practical approximation of the NID called the *Normalized Compression Distance*:

$$\text{NCD}(x, y) := 1 - \frac{L(x) + L(y) - L(xy)}{\max\{L(x), L(y)\}} = \max \left\{ \frac{L(xy) - L(y)}{L(x)}, \frac{L(xy) - L(x)}{L(y)} \right\}. \quad (1)$$

This definition is slightly different from the one in the references, but the difference is negligible; we prefer this formulation because it is easier to interpret.

4 LeanOM

LeanOM is an ontology alignment algorithm that uses the NCD and an assignment problem solver as its basic building blocks. We presently consider only alignment tasks where the output alignment needs to be a bijection between sets of entities from two ontologies O_1 and O_2 , but the method can be generalised, which is what we plan to do in future work.

Let C_1 and C_2 be the subsets of the ontologies that we wish to align; we assumed above that $|C_1| = |C_2|$. Elements of C_1 and C_2 are assumed to be OWL entities (classes, data properties, object properties, or individuals) or SKOS concepts.

To be able to use general purpose data compression software to calculate the NCD, we introduce a representation function R that maps the entities we wish to align to a serialized representation of all information that is deemed relevant to their identification. For example, $R(c)$ may be a serialization of a sub-ontology containing both c and its local context. Defining a suitable mapping is crucial for the algorithm’s performance; our design choices are discussed in more detail in the next section.

We interpret the ontology alignment as an assignment problem [4], where the cost of linking an entity $c \in C_1$ to $c' \in C_2$ is given by their normalized compression distance $\text{NCD } R(c), R(c')$. We then find the assignment that minimizes the total cost using the Hungarian algorithm [26]. Thus, in pseudocode the algorithm looks as follows:

```

Data:  $O_1, O_2$ 
Result: An alignment  $A$  between  $O_1, O_2$ 
for  $c_i \in C_1, c_j \in C_2$  do
  |  $T[i][j] := \text{NCD}(R(c_i), R(c_j));$ 
end
 $A := \text{HungarianAlgorithm}(T);$ 

```

Algorithm 1: LeanOM (NCD function as defined in (1); c_i, c_j as particular cases of c, c')

4.1 Representation function

The compression algorithm used in the calculation of the NCD will be sensitive to the representation of the candidate pairs (c, c') . If c and c' use a different serialization format (like RDF/XML, N-Triples or Turtle), the NCD will be compromised. To avoid this, the implementation must perform some normalization of the candidate pairs. In this paper we propose two normalization methods. First, the entity pairs (c, c') can be normalized if they are retrieved from the Web, for instance via SPARQL queries. The result set of such queries ensures an homogeneous data representation format (commonly JSON, CSV). Second, the implementation can make use of external libraries (*e.g.* Jena, OWL API) that read the input ontologies and allow local in-memory querying of the (c, c') pairs. Then, the pairs can be temporarily serialized using the same format before computing their NCD. In this work we use this last approach, serializing the (c, c') pairs in N-Triples.

It is also important to resolve how much context of an entity c should be included in the serialization $R(c)$. One possible choice is to include only the statements that define such entities, for instance the triple `<http://foo/bar#foobar> rdf:type owl:Class`. On the other hand, the serialization may contain, in addition to the triple defining the type of the entity, all the triples that share the same subject (*i.e.* the list of properties of such entity):

```
<http://foo/bar#foobar> rdf:type owl:Class ;
```



```
skos:prefLabel "bar"@en ;
rdfs:subClassOf <http://foo/bar#baz> .
```

This is the design choice we made in the experiments described in Section 5. Other context selections, like adding all the properties of possibly referred entities (in the last example, adding all the triples that have `<http://foo/bar#baz>` as subject), are also possible; further experiments are required to accurately refine the serialization function R .

5 Experiments and results

We implement LeanOM in order to evaluate its performance⁸, using gzip as a compressor. The full implementation is very short (about 70 lines) due to the simplicity of the approach. It is also fast and generic. Since the SEALS platform is the *de facto* standard for evaluating matchers in the Ontology Matching community, we generate a SEALS package containing LeanOM’s code⁹.

We run this implementation in the SEALS platform, using the OAEI benchmark test library¹⁰. The goal of these tests is to evaluate the strength and weakness of matchers and to measure their progress. The library consists of several datasets, each of them based on a reference ontology of a specific size and domain. Each of these datasets is composed of several individual tests that confront the reference ontology with a modified version of it. These tests are systematically generated, starting from the reference ontology, and discarding, changing or adding ontology statements in order to evaluate how the algorithm behaves in each case. The tests are identified by a number¹¹. We test our LeanOM implementation with the *biblio* (based on a bibliographic ontology [39]), *jerm* (based on a biomedical ontology [40]) and *provenance* (based on a provenance ontology [41]) datasets of the benchmark test library.

Figure 1 shows the performance of LeanOM in the *biblio* dataset tests. The lowest precision and recall are 0.01 and 0.03, while the highest are 1.0 and 1.0, respectively. The highest scores are achieved in tests 224, 232, 236, 237, 241 and 246. The lowest scores are achieved in tests 260, 250, 261 and 266.

Figure 2 shows the performance of LeanOM in the *jerm* dataset tests. The lowest precision and recall are 0.67 and 0.81, while the highest are 1.0 and 1.0, respectively. The highest scores are achieved in tests 224, 232, 236, 237, 241 and 246. The lowest scores are achieved in tests 261, 266 and 251.

Figure 3 shows the performance of LeanOM in the *provenance* dataset tests. The lowest precision and recall are 0.002 and 0.002, while the highest are 1.0 and 1.0, respectively. The highest scores are achieved in tests 224, 232, 236, 237, 241 and 246. The lowest scores are achieved in tests 261, 254, 266 and 260.

⁸<https://github.com/albertmeronyo/ConceptDrift/blob/master/src/compSim2.py>

⁹<http://oaei.ontologymatching.org/2012/seals-eval.html>

¹⁰<http://oaei.ontologymatching.org/2012/benchmarks/index.html>

¹¹For a complete description of the tests, see <http://oaei.ontologymatching.org/2012/benchmarks/index.html>

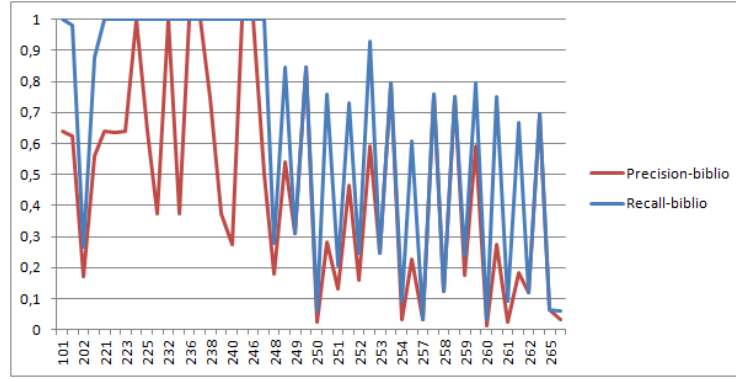


Fig. 1: LeanOM results for the OAEI benchmark test library, *biblio* dataset. The x axis represents the tests performed; the y axis represents precision and recall.

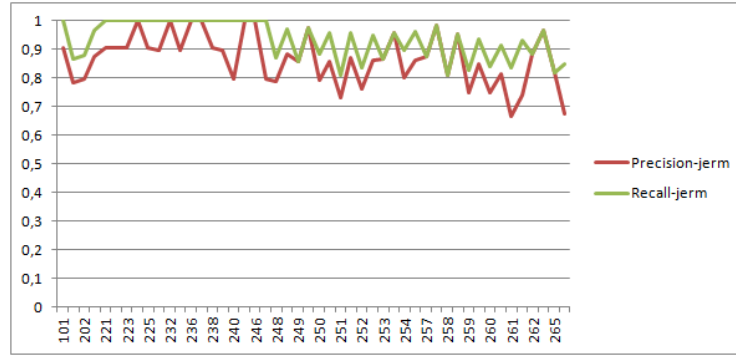


Fig. 2: LeanOM results for the OAEI benchmark test library, *jerm* dataset. The x axis represents the tests performed; the y axis represents precision and recall.

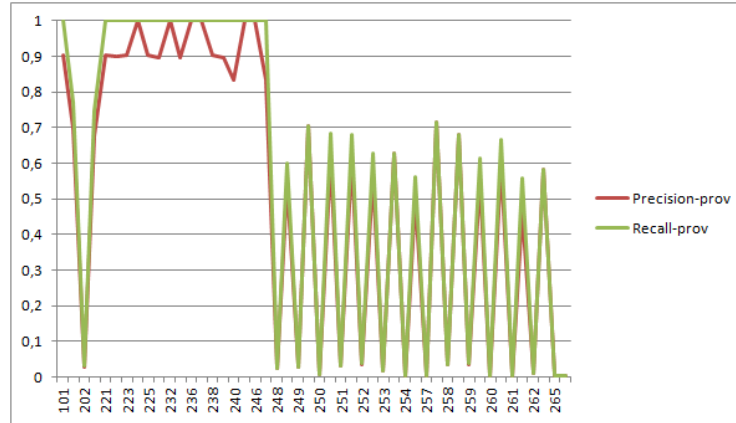


Fig. 3: LeanOM results for the OAEI benchmark test library, *provenance* dataset. The x axis represents the tests performed; the y axis represents precision and recall.

Considering the above, we see that the same tests have the same relative effect on how LeanOM behaves. Perfect scores are achieved in the same tests regardless of the dataset, and lowest scores are achieved also on almost the same tests. Tests from 221 to 247 have perfect recall scores in all the three tests. Likewise, tests 101 and 201 achieve reasonably good precision and recall scores in all cases, while test 202 is revealing some clear weakness of the matcher. All tests from 248 onwards generate the same up and down curve.

The line drawn in all figures has the same overall shape. Hence, we can conclude that the tests have the same impact on the matcher with respect to its strengths and weaknesses, which is consistent with the goal of the benchmark datasets. However, the range of precision and recall rates achieved in each test varies considerably. Figure 1 and 3 show that the worst case in the *biblio* and *provenance* datasets has nearly zero precision and recall, while for the *jerm* dataset, depicted in Figure 2, these figures rarely go beneath 0.8.

Table 1 shows the descriptions of the tests with the highest scores. All tests leave out the individuals of the ontology. 66.67% of the tests flatten the class hierarchies of the ontology. 50.00% of the tests remove the properties (data properties, object properties) of the ontology.

Test number	Description of ontology modification
224	All individuals have been suppressed from the ontology
232	No hierarchy + no instance
236	No instance + no property
237	Flattened hierarchy + no instance
241	No hierarchy + no instance + no property
246	Flattened hierarchy + no instance + no property

Table 1: Description of tests with high scores. We see that tests removing instances and flattening hierarchies are common.

Table 2 shows the descriptions of the tests with the lowest scores. Scrambled labels and the removal of comments are present in all tests. The removal of properties is present in 83.33% of the cases. In 33.33% of the tests the class hierarchy has been expanded. Flattening the class hierarchy, removing it completely and removing ontology instances occurs in 16.67% of the tests.

6 Discussion

Given the results described in Section 5, we can conclude that the performance of LeanOM is situational. First, the presence or absence of ontology instances does not affect the matching process, given that the matcher is able to match perfectly an ontology with a version of itself where individuals have been removed. Second,

Test number	Description of ontology modification
250	Scrambled labels + no comments + no property
251	Scrambled labels + no comments + flattened hierarchy
254	Scrambled labels + no comments + no hierarchy + no property
260	Scrambled labels + no comments + flattened hierarchy + no property
261	Scrambled labels + no comments + expanded hierarchy + no property
266	Scrambled labels + no comments + expanded hierarchy + no instance + no property

Table 2: Description of tests with low scores. We see that tests scrambling labels and removing comments are common.

the matcher performance is very high when the aligned ontologies have essential differences in their class hierarchies, especially when these hierarchies have been completely flattened. Third, precision and recall are also very high regardless of presence or absence of data and object properties, that seem to have a minimal impact during the process. On the other hand, LeanOM is very sensitive to modified labels, and the performance of the matching process is severely affected by this.

The *jerm* dataset shows much higher performance on the same tests than *biblio* and *provenance*. This ontology contains annotations with stable identifiers, that do not get modified during the benchmark process. This fact has several interpretations. On the one hand, the presence of stable identifiers reveals a previous alignment already embedded in the ontologies that facilitates the matching task. On the other hand, this result proves the feature of LeanOM of discovering and exploiting common patterns between the ontologies without an explicit mention on where these patterns are exactly located (annotations, labels, URIs, structural properties, etc.).

Consequently,

- We expect LeanOM to produce good quality mappings in versioned ontologies, ontologies that extend previous ones, or ontologies that model the same domain at different points in time. In these cases, even though class hierarchies may have changed or new individuals may have been added or deleted, LeanOM will be able to detect the commonalities of evolving, extended or versioned entities.
- We expect LeanOM to produce bad quality mappings in different ontologies that model the same domain. If modellers use different terminologies or design strategies, we do not expect LeanOM to align these ontologies properly.

Looking back at the research questions in Section 1, we can conclude that an information complexity approach for ontology matching is good enough to monitor the evolution of a single ontology and, more concretely, of the entity components of an ontology that change over time. With respect to the question of rich serializations of concepts (which are prior to the compression step),

we argue for a *normalized* representation of concepts that takes into account their context. However, we limit this context to the immediately adjacent nodes of the concept type definition to avoid introducing excessive noise to the compressor. The question of how to relate changes in the information distance to changes in semantics is still open. In Section 7 we discuss further work regarding semantically-aware compressors that may constitute a step forward in this direction.

7 Conclusions

In this paper we present the technique of Lean Ontology Matching, which uses an algorithmic notion of information distance, more concretely the Normalized Compression Distance, to quantify the information shared between two different ontologies. We motivate its development in the context of evolving ontologies, presenting the theoretical foundations of Kolmogorov complexity, the Normalized Information Distance, and its approximation using data compression. We present the design of an algorithm, LeanOM, that uses the NCD, and we discuss how the data should be represented for the NCD to produce meaningful distance scores. An experiment set in the SEALS platform is presented, together with results on several benchmarking datasets. We describe these results, emphasizing strong and weak points of LeanOM, and discuss how these translate in different real ontology matching scenarios.

We plan to extend this work in the near future in several ways. First, in order to scale the experimentation framework, we will test additional NCD implementations using other compressors and studying how they behave with respect to reference ontology alignments; we will include more serialization formats and analyze their impact in the distance computation; and we will test other query patterns to explore how different context selections influence these information distances. Second, we will broaden the evaluation work with practical ontology matching tasks, illustrating with real examples the convenience of LeanOM for evolving ontologies. Finally, we plan to investigate other compression techniques that model ontology structural data better.

Acknowledgements The work on which this paper is based has been partly supported by the Computational Humanities Programme of the Royal Netherlands Academy of Arts and Sciences, under the auspices of the CEDAR project. For further information, see <http://ehumanities.nl>.

References

1. Benedetto, D., Caglioti, E., Loreto, V.: Language trees and zipping. *Physical Review Letters* 88(4) (2002)
2. Bennett, C.H., Gács, P., Li, M., Vitányi, P.M.B., Zurek, W.: Information distance. *IEEE Transactions on Information Theory* 44, 1407–1423 (July 1998)

3. Bennett, C., Li, M., Ma, B.: Chain letters and evolutionary histories. *Scientific American* pp. 76–81 (June 2003)
4. Burkard, R., Dell’Amico, M., Martello, S.: *Assignment Problems*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA (2009)
5. Choi, N., Song, I.Y., Han, H.: A survey on ontology mapping. *SIGMOD Record* 35(3), 34–41 (2006)
6. Cilibrasi, R.: *Statistical Inference Through Data Compression*. Ph.D. thesis, University of Amsterdam (2007)
7. Cilibrasi, R., Vitányi, P.M.: Clustering by compression. *IEEE Transactions on Information Theory* 51(4), 1523–1545 (2005)
8. Euzenat, J., Shvaiko, P.: *Ontology matching*. Springer-Verlag, Berlin Heidelberg (DE) (2007)
9. Fanizzi, N., d’Amato, C., Esposito, F.: Conceptual Clustering and Its Application to Concept Drift and Novelty Detection. In: *The Semantic Web: Research and Applications*, 5th European Semantic Web Conference, ESWC 2008. Proceedings. pp. 318–332. ESWC’08, Springer-Verlag, Berlin, Heidelberg (2008)
10. Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web* (2013)
11. Flouris, G., Manakanatas, D., Kondylakis, H., Plexousakis, D., Antoniou, G.: Ontology change: classification and survey. *Knowledge Eng. Review* 23(2), 117–152 (2008)
12. Gonçalves, R.S., Parsia, B., Sattler, U.: Analysing multiple versions of an ontology: A study of the NCI Thesaurus. In: *Proceedings of the 24th International Workshop on Description Logics (DL 2011)* (2011), <http://ceur-ws.org/Vol-745/>
13. Gonçalves, R.S., Parsia, B., Sattler, U.: Facilitating the analysis of ontology differences. In: *Proceedings of the Joint Workshop on Knowledge Evolution and Ontology Dynamics (EvoDyn)* (2011)
14. Haase, P., Harmelen, F., Huang, Z., Stuckenschmidt, H., Sure, Y.: A framework for handling inconsistency in changing ontologies. *The Semantic Web–ISWC 2005* pp. 353–367 (2005)
15. Heflin, J., Pan, Z.: A model theoretic semantics for ontology versioning. In: *In Third International Semantic Web Conference*. pp. 62–76. Springer (2004)
16. Huang, Z., Stuckenschmidt, H.: Reasoning with multi-version ontologies: A temporal logic approach. In: *In Proceeding of the 4th International Semantic Web Conference (ISWC)*. pp. 398–412 (2005)
17. Huber, J., Sztyler, T., Noessner, J., Meilicke, C.: CODI: Combinatorial Optimization for Data Integration. Results for OAEI 2011 (2011), <http://ceur-ws.org/Vol-814>
18. Ichise, R., Takeda, H., Honiden, S.: Integrating multiple internet directories by instance-based learning. In: *Proceedings of the eighteenth International Joint Conference on Artificial Intelligence* (2003)
19. Isaac, A., van der Meij, L., Schlobach, S., Wang, S.: An empirical study of instance-based ontology matching. In: *Proceedings of the the 6th International Semantic Web Conference (ISWC 2007)*. Lecture Notes in Computer Science, vol. 4825, pp. 253–266. Springer (2007)
20. Jean-Mary, Y.R., Shironoshita, E.P., Kabuka, M.R.: Ontology matching with semantic verification. *Web Semantics: Science, Services and Agents on the World Wide Web* 7(3), 235 – 251 (2009)
21. Kalfoglou, Y., Schorlemmer, M.: Ontology mapping: the state of the art. *The knowledge engineering review* 18(01), 1–31 (2003)

22. Kalyanpur, A., Parsia, B., Sirin, E., Grau, B., Hendler, J.: Swoop: A web ontology editing browser. *Web Semantics: Science, Services and Agents on the World Wide Web* 4(2), 144–153 (2006)
23. Klein, M.: Change Management for Distributed Ontologies. Ph.D. thesis, Vrije Universiteit Amsterdam (Aug 2004), <http://www.cs.vu.nl/~mcaklein/thesis/>
24. Klein, M.C.A., Fensel, D., Kiryakov, A., Ognyanov, D.: Ontology versioning and change detection on the web. In: *Proceedings of the 13th International Conference on Knowledge Engineering and Knowledge Management. Ontologies and the Semantic Web*. pp. 197–212. EKAW '02, Springer-Verlag, London, UK (2002), <http://portal.acm.org/citation.cfm?id=645362.756436>
25. Klenner, M., Hahn, U.: Concept versioning: A methodology for tracking evolutionary concept drift in dynamic concept systems. In: *In Proc. of ECAI 1994*. pp. 473–477. Wiley (1994)
26. Kuhn, H.W.: The Hungarian Method for the assignment problem. *Naval Research Logistics Quarterly* 2, 83–97 (1955)
27. Kumar, S.K., Harding, J.: Ontology mapping using description logic and bridging axioms. *Computers in Industry* 64(1), 19 – 28 (2013), <http://www.sciencedirect.com/science/article/pii/S0166361512001418>
28. Leme, L.A.P.P., Casanova, M.A., Breitman, K.K., Furtado, A.L.: Instance-based OWL schema matching. In: Filipe, J., Cordeiro, J. (eds.) *ICEIS. Lecture Notes in Business Information Processing*, vol. 24, pp. 14–26. Springer (2009)
29. Li, J., Tang, J., Li, Y., Luo, Q.: Rimom: A dynamic multistrategy ontology alignment framework. *IEEE Trans. Knowl. Data Eng.* 21(8), 1218–1232 (2009)
30. Li, M., Chen, X., Li, X., Ma, B., Vitányi, P.: The similarity metric. *IEEE Transactions on Information Theory* 50(12), 3250–3264 (2004)
31. Li, M., Vitányi, P.M.: *An Introduction to Kolmogorov Complexity and its Applications*. Springer-Verlag, New York, third edn. (2008)
32. Li, W.S., Clifton, C., Liu, S.Y.: Database integration using neural networks: implementation and experiences. *Knowledge and Information Systems* 2, 73–96 (2000)
33. Liang, Y., Alani, H., Shadbolt, N.: Changing ontology breaks queries. In: Cruz, I., Decker, S., Allemang, D., Preist, C., Schwabe, D., Mika, P., Uschold, M., Aroyo, L. (eds.) *The Semantic Web - ISWC 2006, Lecture Notes in Computer Science*, vol. 4273, pp. 982–985. Springer Berlin / Heidelberg (2006), http://dx.doi.org/10.1007/11926078_79
34. Mao, M., Peng, Y., Spring, M.: An adaptive ontology mapping approach with neural network based constraint satisfaction. *Web Semantics: Science, Services and Agents on the World Wide Web* 8(1), 14 – 25 (2010), <http://www.sciencedirect.com/science/article/pii/S1570826809000717>
35. Mascardi, V., Locoro, A., Rosso, P.: Automatic Ontology Matching via Upper Ontologies: A Systematic Evaluation. *Knowledge and Data Engineering, IEEE Transactions on* 22(5), 609–623 (2010)
36. Meroño-Peñuela, A., Schlobach, S., van Harmelen, F.: Semantic Web for the Humanities. In: *Semantics and Big Data: Proceedings of ESWC 2013. LNCS 7882*. pp. 645–649. Springer-Verlag, Berlin, Heidelberg (2013)
37. Noy, N.F., Klein, M.: Ontology evolution: Not the same as schema evolution. *Knowledge and Information Systems* 6, 428–440 (2004), <http://dx.doi.org/10.1007/s10115-003-0137-2>
38. Noy, N.F., Musen, M.A.: Promptdiff: a fixed-point algorithm for comparing ontology versions. In: *Eighteenth national conference on Artificial intelligence*. pp. 744–750. American Association for Artificial Intelligence, Menlo Park, CA, USA (2002), <http://portal.acm.org/citation.cfm?id=777092.777207>

39. Ontology Alignment Evaluation Initiative: The Bibliographic Ontology (2012), <http://oeai.ontologymatching.org/2011/benchmarks/101/onto.rdf>
40. Ontology Alignment Evaluation Initiative: The Jerm Ontology (2012), <http://oeai.ontologymatching.org/2012/benchmarks/jerm.owl>
41. Ontology Alignment Evaluation Initiative: The Provenance Ontology (2012), <http://oeai.ontologymatching.org/2012/benchmarks/provenance.owl>
42. Plessers, P., Troyer, O.D., Casteleyn, S.: Understanding ontology evolution: A change detection approach. *Web Semantics: Science, Services and Agents on the World Wide Web* 5(1), 39 – 49 (2007), <http://www.sciencedirect.com/science/article/B758F-4MX4VXJ-1/2/a6eaf2835f9d1ca7799bcc27980d2973>, selected Papers from the International Semantic Web Conference (ISWC2005)
43. Santos, C.C., Bernardes, J., Vitányi, P., Antunes, L.: Clustering fetal heart rate tracings by compression. In: *Proceedings of the 19th IEEE Symposium on Computer-Based Medical Systems*. pp. 685–690 (2006)
44. Shvaiko, P., Euzenat, J.: Ontology matching: State of the art and future challenges. *IEEE Transactions on Knowledge and Data Engineering* 25(1), 158–176 (2013)
45. Sure, Y., Erdmann, M., Angele, J., Staab, S., Studer, R., Wenke, D.: OntoEdit: Collaborative ontology development for the semantic web. *The Semantic Web?ISWC 2002* pp. 221–235 (2002)
46. Teahan, W.J.: Text classification and segmentation using minimum cross-entropy. In: *Proceeding of RIAO-00, 6th International Conference on Content-Based Multimedia Information Access* (2000)
47. Thor, A., Kirsten, T., Rahm, E.: Instance-based matching of hierarchical ontologies. In: Kemper, A., Schöning, H., Rose, T., Jarke, M., Seidl, T., Quix, C., Brochhaus, C. (eds.) *BTW. LNI*, vol. 103, pp. 436–448. GI (2007)
48. Todorov, K., Geibel, P., Kühnberger, K.U.: Mining concept similarities for heterogeneous ontologies. In: Perner, P. (ed.) *ICDM. Lecture Notes in Computer Science*, vol. 6171, pp. 86–100. Springer (2010)
49. Tsymbol, A.: The problem of concept drift: definitions and related work. Tech. Rep. TCD-CS-2004-15, Computer Science Department, Trinity College Dublin, Ireland (2004)
50. Udrea, O., Getoor, L., Miller, R.J.: Leveraging data and structure in ontology integration. In: Chan, C.Y., Ooi, B.C., Zhou, A. (eds.) *SIGMOD Conference*. pp. 449–460. ACM (2007)
51. Wang, S., Englebienne, G., Schlobach, S.: Learning concept mappings from instance similarity. In: *Proceedings of the 7th International Semantic Web Conference (ISWC 2008)*. Karlsruhe, Germany (2008)
52. Wang, S., Schlobach, S., Klein, M.C.A.: What Is Concept Drift and How to Measure It? In: *Knowledge Engineering and Management by the Masses - 17th International Conference, EKAW 2010. Proceedings*. pp. 241–256. *Lecture Notes in Computer Science*, 6317, Springer (2010)
53. Wang, S., Schlobach, S., Klein, M.C.A.: Concept drift and how to identify it. *J. Web Sem.* 9(3), 247–265 (2011)
54. Wartena, C., Brussee, R.: Instance-based mapping between thesauri and folksonomies. In: *ISWC'08* (2008)
55. Zaiss, K.S.: Instance-based ontology matching and the evaluation of matching systems. Ph.D. thesis, Heinrich Heine Universität Düsseldorf (2010)